

SQL MINI PROJECT | ADDA247

Retail Store SQL Analysis

Reports and insights from 42 SQL queries across 6 tables

Arijit Dutta



The business at a glance

Headline numbers from the orders table



400

Total orders placed



30

Active customers



₹69.6L

Total order value (all statuses)



₹17,402

Average order value



Insight: the average customer placed about 13 orders ($400 / 30$) worth roughly ₹2.3 lakh in total, so customers here buy repeatedly.

Six tables, one story

The retail store database behind the analysis



customers

Name, email, phone and sign-up date



products

Category, price, stock quantity and date added



orders

Order date, status, total amount and customer



order_items

Product, quantity and item price per order line



payments

Payment method, amount paid and payment date



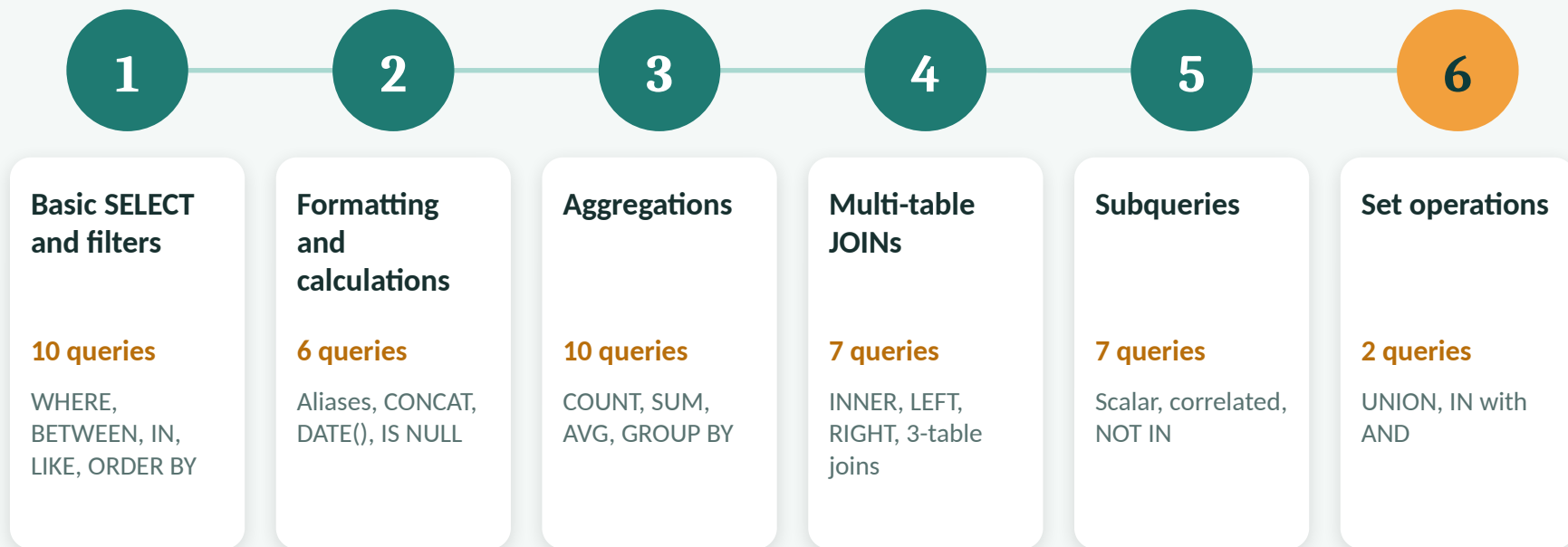
product_reviews

Reviews written by customers

How they connect: customers → orders → order_items → products | orders → payments | customers → product_reviews

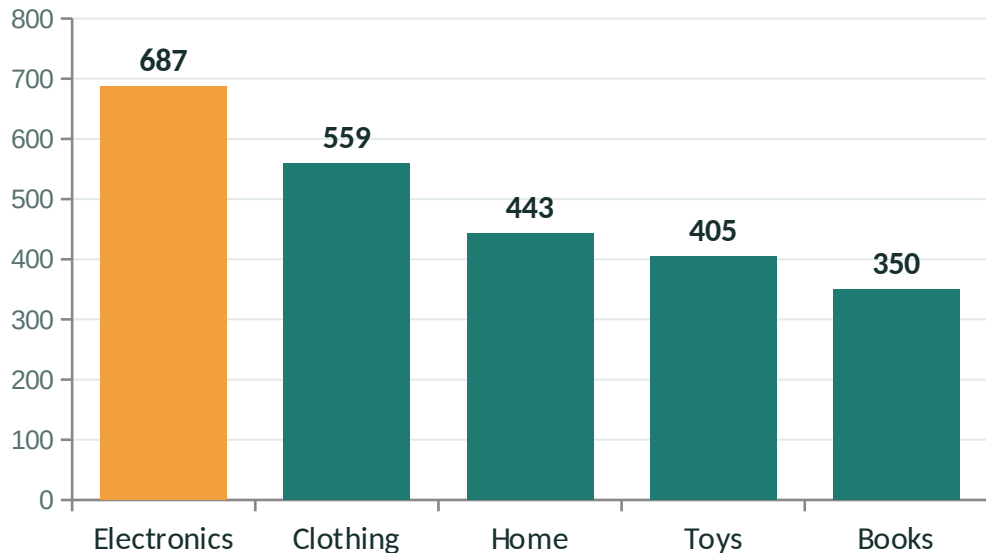
42 queries, six levels

Each level added a new SQL skill



Electronics drives sales volume

Units sold per product category



Source: Level 3, Question 7

28%

of all 2,444 units sold are Electronics (687 units).

Books trail at 350 units (14%), so the top category sells almost 2x the bottom one.

Clothing is priciest, Home is cheapest

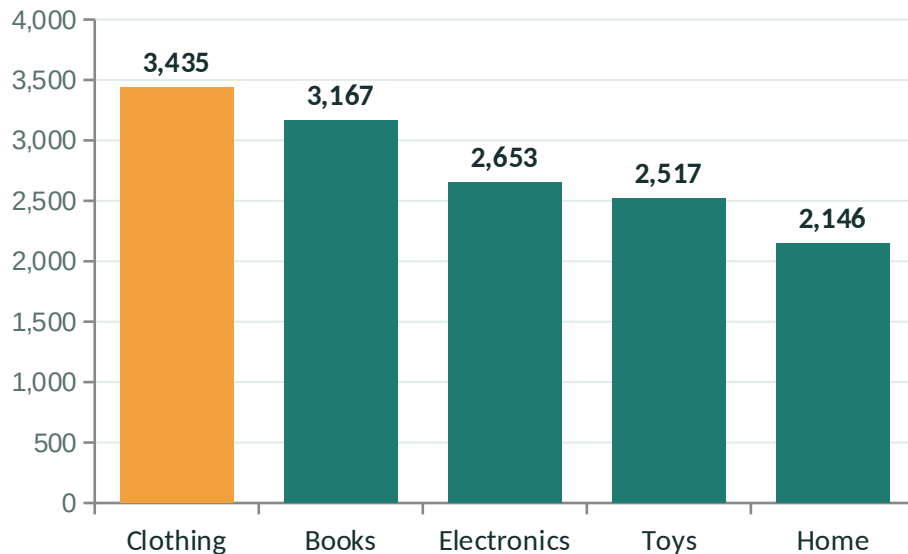
Average product price per category

Price is not the whole story

Clothing has the highest average price yet is second in units sold (559). Books, second-priciest, sell the fewest (350).

The volume leader is mid-priced

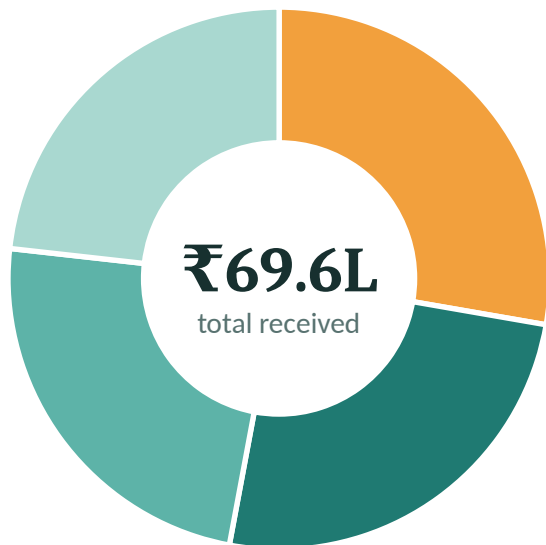
Electronics sells the most units at an average of about ₹2,653, right in the middle of the price range.



Average of catalogue prices in ₹, not weighted by sales. Source: Level 3, Question 8

Payments split almost evenly

Total payments received per payment method



Debit Card ₹19.31L 27.7%

Credit Card ₹17.55L 25.2%

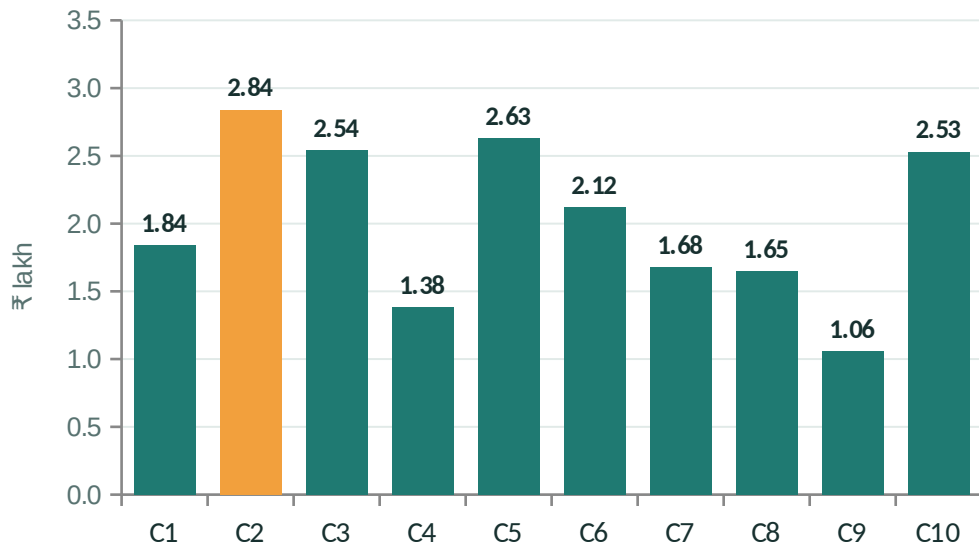
Net Banking ₹16.58L 23.8%

UPI ₹16.17L 23.2%

Insight: only about 4.5 points separate first from last, so no single method dominates.

Customer value varies widely

Total spend per customer, customers 1 to 10



First 10 customers from the Level 3 Q6 output. Totals include all order statuses.

13.3

average orders per customer (400 orders / 30 customers)

₹2.32L

average spend per customer across all 30

₹42,056

largest single order among customers 1 to 10 (customer 2)

Data health check: all clear

Five checks, five clean results



0

orders with a
missing customer

Level 2 · Q1



0

products out of
stock

Level 2 · Q6



0

customers who
never ordered

Level 5 · Q4



0

products that were
never ordered

Level 5 · Q5



₹0

gap between
payments and order
value

Level 3 · Q2 and Q10

Why it matters: clean keys and matching totals mean the earlier numbers can be trusted, with one caveat next.

Technique spotlight: correlated subquery

Orders above that customer's own average

```
SELECT a.order_id, a.order_date,  
       a.customer_id, a.total_amount  
FROM orders a  
WHERE a.total_amount > (  
    SELECT AVG(b.total_amount)  
    FROM orders b  
    WHERE a.customer_id = b.customer_id  
);
```

Source: Level 5, Question 3

1

Outer query

Looks at every order, aliased as a.

2

Inner query

Recalculates the average for that order's own customer (b).

3

Filter

Keeps only the orders that beat their customer's average.

What I'd tighten, and what's next

Honest caveats and the next skills to add



What I'd tighten

Revenue counts every status. Cancelled and Pending orders are included; filter to fulfilled orders for true revenue.

NOT IN needs a NULL guard. I added IS NOT NULL; NOT EXISTS is the safer pattern.

Ties return extra rows. MAX equality can return several orders per customer; ROW_NUMBER() returns one.



What's next

Window functions. RANK and running totals per customer.

CTEs and time trends. Monthly revenue and order growth.

Power BI dashboard. Turn these queries into an interactive report.

Thank you

Full queries, output screenshots and a README are on GitHub. Questions and feedback are welcome.



github.com/arijitddutta



[linkedin.com/in/arijit-dutta-552b41199](https://www.linkedin.com/in/arijit-dutta-552b41199)

